

KVM-Based eBPF Rootkit Detection Engine

Milestone one

Luis Abraham - labrahamesco2024@my.fit.edu

Dylin Irons - dirons2024@my.fit.edu

Dominick Morales - dmorales2024@my.fit.edu

Braiden Ames - bames2024@my.fit.edu

Dr. Ribeiro - eribeiro@fit.edu

Client: Amy Alvarez, NASA Scientist

Milestone Completion Matrix:

Task	Completion	Luis	Braiden	Dominick	Dylin
Develop tooling for easy VM setup (CLI, JSON configs, integration with introspection tooling, and more)	100%	Write 10%	Write 45%	Write 45%	Write 0%
Develop web server for users	100%	Write 0%	Write 0%	Write 0%	Write 100%
Write LibVM introspection back end	100%	Write 100%	Write 0%	Write 0%	Write 0%
Requirement Document	100%	Write 25%	Write 25%	Write 25%	Write 25%
Design Document	100%	Write 25%	Write 25%	Write 25%	Write 25%

Test Plan	100%	Write 25%	Write 25%	Write 25%	Write 25%
-----------	------	-----------	-----------	-----------	-----------

Discussion of each accomplished task:

1. QEMU/KVM was selected as our hypervisor, and an initial VM management CLI (create, start, stop, restart, inspect, destroy) was built and tested to serve as the foundation for VM setup
2. So far the entire frontend of the RootView site is built. The server, data, detection rules, and event pipeline are being tested with testing data until the LibVMI engine is up and running to get data from an actual device.
3. An Ubuntu 22.04 Virtual machine image has been created and set up. This virtual machine image has a compiled kvm-vmi kernel, modified QEMU 4.2.1, docker, and upstream libvmi so that the user can begin introspecting on nested virtual machines whether it is through RootView or other custom programs. The reasoning behind this virtual machine is that kvm-vmi requires a modified kernel in order to allow for introspection in the first place. However, the newest kernel sources provided by kvm-vmi range from kernel 5.4 to 5.14, which we would not like to be installing on our host machines. In the future, we may experiment with making these changes to newer versions of the Linux kernel and report on our findings if it works. The end result is that a user can visit our github page, download the image, and use it with their local QEMU provided that the machine supports nested virtualization.
4. Initial pKVM introspection tools were created in order to gauge expanding to devices other than x86 devices. Not many ARM64 consumer devices support virtualization, but some Google Pixel devices do via pKVM, which allows us to test a wider range of devices and experiment with creating new introspection tooling. Working with pKVM is slightly counterintuitive as the purpose behind pKVM is to protect the information and data inside a VM from outside viewers. It is still possible to make this work by developing EL2 modules, but it is strictly for research devices. The next step would be to expand beyond this initial foothold and introspect into a pKVM VM in a similar manner to what we are doing with kvm-vmi.
5. The requirements document was written to include an overview of the project, featuring the problem we are trying to solve, operating equipment, design implementations and assumptions we make. Also featured in the document is external requirements and functional requirements. This document keeps us in track for ensuring all needs of the tool are met.
6. For the design document it highlights the structure of Rootview. Within the document we laid out pKVM design, Rootview detection engine, VMI abstraction, eBPF introspection, and exit monitoring. To follow up is the front end

of the tool, the web server. The UI is designed to have a user facing dashboard to control the tool and keep it user friendly.

7. The testing document lays out functional testing and expected results for the tool. Ensures that the design and development meets the expected outcome for the tools.

Individual Accomplishments:

- **Dominick:** Set up a VM with KVM (a KVM fork that allows for introspection) flashed into its kernel, then made it so a user or developer can just download it from the “GitHub Releases” in the repo and have a very easy setup.
- **Luis:** Since there are no VMI tools for instrumenting pKVM, he started to develop another toolkit that will allow for that. For now it is a simple “hello world” from EL2 and a few development and setup scripts. Also set up VM exit hooks.
- **Braiden:** Wrote a VM lifecycle test harness in C that exercises create, list, inspect, start, stop, restart, and destroy operations against the VM subsystem, with pass/fail assertions and duplicate-name/invalid-state rejection checks.
- **Dylin:** Setup webserver and landing page website for the Rootview dashboard. Backend will be completed after the LibVMI engine is functioning on a device to send data to the website.

Next Milestone Matrix:

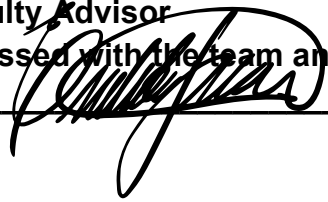
Task	Luis	Braiden	Dominick	Dylin
pKVM module VMI	100%	0%	0%	0%
Linux kernel introspection	10%	90%	0%	0%
Initial eBPF introspection	10%	0%	90%	0%
Initial Python bindings	0%	0%	0%	100%
Bindings integration w/ web UI	0%	0%	0%	100%

Discussion of each planned task:

1. Develop similar tooling in pKVM to what we used with KVM to set up reading memory, registers, and maybe exit hooks.

2. Map **kallsyms** to virtual memory during a paused state, get process listings (task_struct), analyzing other important objects, and more to aid future eBPF analysis
3. Analyze basic eBPF info like: loaded eBPF programs and basic metadata, eBPF maps and their metadata, program types, IDs, names, and attachment information
4. Create python bindings to milestone 1 work like: VM handling, phys/virt mem/reg r/w, exit hooks, etc. We have CLI tooling for it, so you'd just need to use similar logic.
5. Integrate those python bindings into the web server: VM management and advances introspection settings (this is where you can add the KVM stuff)

Approval from Faculty Advisor

- "I have discussed with the team and approved this milestone."
- Signature:  _____ Date: 09/24/2026